```
    case task_complete:
         task_thread.join();
         display_results();
         break;
    default:
         //...
    }
}
```

By separating the concerns in this way, the user thread is always able to respond to the events in a timely fashion, even if the task takes a long time. This *responsiveness* is often key to the user experience when using an application; applications that completely lock up whenever a particular operation is being performed (whatever that may be) are inconvenient to use. By providing a dedicated event-handling thread, the GUI can handle GUI-specific messages (such as resizing or repainting the window) without interrupting the execution of the time-consuming processing, while still passing on the relevant messages where they *do* affect the long-running task.

So far in this chapter you've had a thorough look at the issues that need to be considered when designing concurrent code. Taken as a whole, these can be quite overwhelming, but as you get used to working with your "multithreaded programming hat" on, most of them will become second nature. If these considerations are new to you, hopefully they'll become clearer as you look at how they impact some concrete examples of multithreaded code.

8.5. Designing concurrent code in practice

When designing concurrent code for a particular task, the extent to which you'll need to consider each of the issues described previously will depend on the task. To demonstrate how they apply, we'll look at the implementation of parallel versions of three functions from the C++ Standard Library. This will give you a familiar basis on which to build, while providing a platform for looking at the issues. As a bonus, we'll also have usable implementations of the functions, which could be used to help with parallelizing a larger task.